Senior DevOps Engineer

Skills Assessment - Response

TechnologyAdvice

Take-Home Case Study

Prepared by: Vipul Mayekar

AWS Certified Solutions Architect - Associate | Cloud & DevOps Engineer

vipulm.tech5@gmail.com | +91 98202 91063 | Mumbai, India

Reliability - Deployment - Provisioning - Cloud Cost - Observability

Contents

Executive Summary

Deliverable 1 - Prioritisation: which problems I address first, and why

Deliverable 2 - Incident Investigation: the OOMKill / memory incident

Deliverable 3 - Systemic Improvement: self-service deployment safety

Deliverable 4 - Technical Artifact: GitHub Actions + ArgoCD progressive delivery

Deliverable 5 - Use of AI during this assessment

Deliverable 6 - Slack update to the VP of Technology

Appendix - Supporting artifacts (HPA/VPA, PrometheusRule, cost guardrails)

How to read this. The six deliverables are answered in order. Where I make a recommendation I state the trade-off and what I would verify before acting, because leadership asked me to decide where to focus first, not to boil the ocean. Examples are drawn from systems I have actually built and operated (EKS with KEDA/Karpenter, Jenkins/GOCD CI/CD, Terraform IaC, ELK/OpenTelemetry observability, and a ~$30K/month AWS cost-optimisation program).

Executive Summary

Across my first few weeks I found five recurring problems: production reliability (repeat incidents, most recently an OOMKill cascade), a partly-manual deployment process, slow environment provisioning, a 25% cloud-cost rise against only 8% traffic growth, and organically-grown observability causing alert fatigue and customer-first incident detection.

My framing is deliberate: these are not five independent fires. Two of them - observability and reliability - are the same root cause wearing different clothes. We learned about the last incident from customers, and the alert fired after errors had already climbed. So my first move is not to chase every symptom; it is to make the system tell us the truth earlier and to stop the known failure mode from recurring. Everything else is sequenced behind that.

One-line thesis. Fix detection and the known reliability failure first (highest customer-facing risk, lowest effort), then attack cost (clear financial signal) and deployment toil (recurring team drain) in parallel, and treat slow provisioning as an important-but-deferrable platform investment.

Deliverable 1 - Prioritisation

Leadership asked where I would focus first, what I would defer, and what I would gather before deciding. I rank on four axes: customer/revenue risk, blast radius, effort-to-impact, and whether the problem compounds if ignored.

The ranking

Rank 1 (now) - Observability & detection, plus the specific OOM reliability fix

- **Why first:** Incidents are being found by customers, and the one alert we had lagged the impact. Until detection is trustworthy, every other improvement is flying blind. This is the highest customer risk and, for the detection piece, relatively low effort (tune SLO-based alerts, add leading indicators).
- **Paired reliability fix:** Implement a right-sized memory signal and resource requests/limits so the exact failure mode that caused the last incident cannot silently recur. This is a contained, high-confidence fix.

Effort/impact: High impact, low-to-medium effort. Mostly configuration and SLO definition, not new platforms.

Rank 2 (now, parallel track) - Cloud cost

- **Why high:** Spend up ~25% while traffic up ~8% is a measurable, compounding leak. Every month unexamined is money gone, and the signal is unambiguous enough to start today.
- **Why parallel:** Cost investigation is largely read-only (Cost Explorer, CUR, tagging) so it runs safely alongside the reliability work without competing for the same change windows.

Rank 3 (next) - Deployment process toil

- **Why not first:** 6-8 DevOps hours/week is a real, recurring drain and a source of failed-deploy risk, but it is internal pain rather than active customer outage. It is the right next systemic project once detection is trustworthy (Deliverable 3 proposes the fix).

Rank 4 (deferred, but scheduled) - Environment provisioning

- **Why deferred:** 1-2 days per new environment is slow and copy-paste Terraform is risky, but it is a periodic cost, not a daily one, and does not directly threaten live customers. It is a platform-engineering investment (Terraform modules / self-service) best done deliberately, not rushed.

What I would intentionally defer - and say so out loud

- A full platform / internal developer portal rebuild. High value eventually, but it competes with stabilising production. Deferred until reliability and cost are under control.
- A wholesale Datadog monitor rewrite. I will fix the few signals that matter now (SLOs, the OOM class) rather than re-architect all alerting in week one.
- Multi-cloud or large re-platforming ideas. Out of scope for the stated problems; they would add risk without addressing any current pain.

What I would gather before committing

1. **Reliability:** Incident history for all three recent incidents - timelines, blast radius, and whether they share a root cause (are we fixing one problem or three?).
2. **Observability:** Current SLOs/SLIs and the existing Datadog monitor inventory - how many alerts, how many actionable, alert-to-incident ratio, and which pages are noise.
3. **Cost:** A Cost & Usage Report broken down by service, team, and tag; a tagging-coverage audit; and the before/after of any recent architecture or traffic-shape change.
4. **Deployment:** The production release runbook and failed-deploy tickets - which manual steps fail most, and which are automatable versus genuinely needing judgement.
5. **Provisioning:** How often new environments are actually requested, and how much the existing Terraform varies between services (drives module vs. template design).

Judgement note. If the cost investigation surfaced an active runaway (e.g. an exponential data-egress or storage leak), I would promote it to Rank 1 immediately. Priorities are a current best estimate, not a contract - I re-rank as data arrives.

Deliverable 2 - Incident Investigation

The facts: memory climbed gradually for ~90 minutes, pods hit their limit and were OOMKilled, Kubernetes restarted them, error rate spiked during the restart cycle, the memory alert fired only after customers were already impacted, doubling the limit stabilised it, and there was no deployment immediately before. I will treat this as a structured investigation, not a guess-and-restart.

Reading the signal first

A gradual 90-minute climb that ends in OOMKill, with no preceding deploy, is the classic shape of a resource leak or unbounded growth under steady load - not a sudden code regression. Doubling memory did not fix the cause; it bought more time before the same ceiling is hit. That distinction drives the whole investigation: I am looking for why memory grows without bound, not merely how to survive it.

Timeline of the incident

Figure 2.1 - What happened, and where detection lagged

```
T+0min    Memory starts climbing gradually (steady traffic, no deploy)

~T+90min  Pods reach memory limit  ->  OOMKilled by kernel/cgroup
~T+90min  Kubernetes restarts containers; connections drop
T+90..?   Error rate spikes during restart/cold-start cycle
(late)    Memory alert finally fires - AFTER customer impact
+restore  Engineer doubles memory limit  ->  app stabilises
after     No permanent follow-up beyond the bigger limit
```

Hypotheses, ranked by how well they fit a gradual climb

H1 (most likely) - Application memory leak / unbounded growth

- **Why it fits:** A leak (un-freed objects, growing in-memory cache, unbounded queue/connection pool, accumulating sessions) grows heap steadily under normal traffic until the cgroup limit is hit. Fits the slow climb and the no-deploy fact perfectly.
- **What I'd expect to see:** Heap/RSS trending up monotonically and not recovering after GC; for a JVM, old-gen filling and full-GC churn near the end.

H2 - Misconfigured runtime / limit interaction (very common in K8s)

- **Why it fits:** A JVM (or Node/Go) that isn't container-memory-aware sizes its heap to the node, not the cgroup limit, so it overshoots and gets OOMKilled. Or the limit was simply set too low for real working-set and headroom.
- **From experience:** I have hit exactly this on EKS - the fix is container-aware heap settings and right-sized requests/limits, not just a bigger number.

H3 - Traffic / workload change (no code deploy needed)

- **Why it fits:** A gradual load increase, a slow-building batch/async job, a large or pathological request (big payloads, expensive query results held in memory), or a retry storm can grow memory without any deployment.

H4 - Downstream slowness causing buffering

- **Why it fits:** If a dependency (DB, cache, external API) slowed down, in-flight requests and buffers pile up in memory while waiting - memory grows as a symptom of a downstream problem, not a leak in this service.

Note on 'doubling the limit worked'. That is consistent with H1-H4 all at once; a higher ceiling delays OOM regardless of cause. It is evidence the limit was the binding constraint, not evidence we found the root cause. I would explicitly resist closing the incident here.

Data I would want

- **Memory metrics:** Per-pod container memory (working set vs. limit), and whether every restart starts low and climbs the same way (classic leak fingerprint).
- **Runtime internals:** For the JVM/runtime: heap vs. non-heap, GC frequency/pause, old-gen occupancy. A heap dump near the ceiling is the single most valuable artifact for H1.
- **Kubernetes state:** kubectl describe / kubectl get events for OOMKilled reason and restart counts; node dmesg -T | grep -i oom to confirm cgroup OOM vs. node pressure.
- **Traffic & latency:** RPS, request-size distribution, latency, and error type over the 90 minutes - did load or payload shape change? Any retry amplification?
- **Downstream health:** Latency/error of DB, cache, and external dependencies over the same window (tests H4).
- **Change log:** Deploy/config history to firmly rule out a lagged change (feature flag flip, config map, dependency rollout).

How I'd investigate - the sequence

6. Confirm the mechanism: verify it was a cgroup OOMKill (exit code 137, reason OOMKilled) on the container, not node-level eviction. This tells me it's the pod's own limit.
7. Pull the memory curve across several restarts. Identical sawtooth from a low baseline each time strongly implicates a leak/unbounded growth (H1).
8. Overlay traffic and downstream latency on the memory curve. If memory tracks a load or latency change, pivot to H3/H4; if it climbs under flat load, stay on H1/H2.
9. Inspect runtime config: is the heap container-aware? Are requests/limits sane versus real working set? (H2)
10. If H1 holds, capture a heap dump near the ceiling and analyse the dominant retained objects - that usually names the leak directly.
11. Only then decide the permanent fix, and reproduce it in a lower environment if possible before rolling out.

Improvements after root cause

Prevent the failure mode

- **Code/config:** Fix the leak / unbounded structure (bounded caches and queues, connection-pool caps, pagination of large results), or correct the runtime so it respects the cgroup limit.

- **Right-size:** Set memory requests and limits from measured working set with headroom. Consider VPA in recommendation mode to keep sizing honest over time.

Detect earlier - this is the part that actually failed

- **Leading signals:** Alert on the leading indicator - memory as a percentage of limit trending up (e.g. >80% for N minutes) and the OOMKill/restart count - not just a late absolute threshold. Page before customers feel it.
- **SLO alerting:** Add an SLO-based burn-rate alert on customer error rate so detection is tied to user experience, not just infrastructure counters.

Reduce blast radius while leaked

- **Resilience:** Readiness probes and PodDisruptionBudgets so restarts drain gracefully and don't all cycle at once; tuned HPA so load spikes scale out rather than fill memory.

Learn from it

- **Process:** Blameless post-incident review with a tracked action item - the scenario explicitly notes there was no permanent follow-up, which is the real organisational gap.

Deliverable 3 - Systemic Improvement

I choose the deployment process. It is the clearest case of recurring, repeated DevOps intervention - 6-8 hours every week of the team being pulled in to rescue failing deployments and walk engineers through manual runbook steps. That is toil by definition: manual, repetitive, automatable, and it scales with the number of teams rather than delivering lasting value.

Current problem

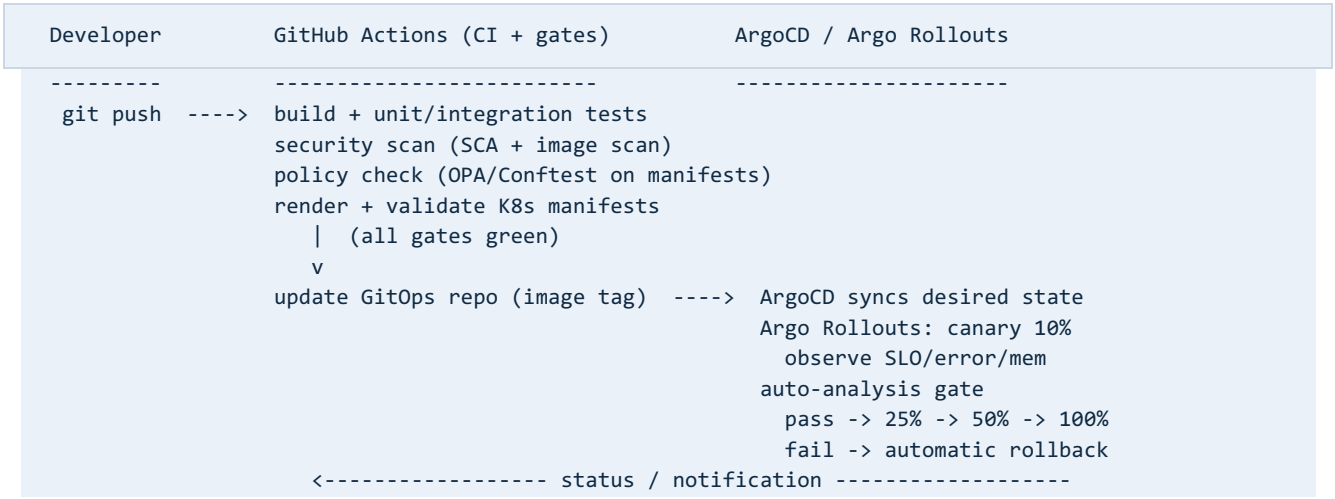
- **Partial automation:** Deploys go through GitHub Actions and ArgoCD and are mostly automated, but production releases still need several manual runbook steps.
- **The gap:** Those manual steps are where deploys fail; engineers then ask DevOps for help, costing the team 6-8 hrs/week and making DevOps a bottleneck on every team's release.
- **Hidden risk:** Manual steps are inconsistent between people and releases, which is also a reliability and audit risk, not only a time cost.

Proposed solution - a paved-road, self-service deployment with safety built in

Make the safe path the easy path. Encode the manual runbook steps as automated gates inside a standardised GitHub Actions + ArgoCD pipeline, and replace the risky manual 'flip to prod' with an automated progressive rollout (canary) that promotes or rolls back on real health signals. Engineers self-serve a production release; DevOps owns the pipeline, not each deploy.

How it would work

Figure 3.1 - Manual runbook steps become automated pipeline gates + progressive delivery



- **Codify the runbook:** Every former manual step (checks, approvals, smoke tests, config validation) becomes a codified, repeatable stage. Required reviewers/approvals become GitHub environment protection rules, so human judgement is kept where it adds value and removed where it was just toil.
- **Progressive delivery:** Argo Rollouts promotes the release in stages, watching error rate and the memory/SLO signals from Deliverable 2. Healthy -> auto-promote; unhealthy -> auto-rollback to the last good revision. No 2 AM manual rollback.

- **Self-service template:** A reusable workflow / template means a new service adopts the paved road by reference, not by copy-paste - which also starts solving the provisioning problem (Deliverable 1, Rank 4).

Risks / trade-offs

- **Change fatigue:** Teams moving from a manual runbook to an automated gated pipeline need buy-in and training. Mitigate by piloting with one willing team and keeping the old path available during transition.
- **Automation trust:** A canary that promotes on bad signals is dangerous. This is why Deliverable 2's detection work is a prerequisite - the auto-analysis is only as trustworthy as the SLOs behind it.
- **Not one-size-fits-all:** Canary needs enough traffic and good metrics to be meaningful; for low-traffic services I'd use blue-green with a smoke-test gate instead.
- **Added complexity:** Progressive delivery and GitOps add moving parts. The payoff is fewer humans in the critical path, but the platform itself must be well-run.

How I'd roll it out safely

12. **Weeks 1-2:** Shadow the current process; document every manual runbook step and classify each as automatable gate vs. genuine human approval.
13. **Weeks 3-4:** Pilot the paved-road pipeline with one lower-risk service in staging, then its production with canary + auto-rollback behind a feature flag.
14. **Weeks 5-8:** Onboard 2-3 more services, fix friction, publish the reusable workflow and docs. Keep the legacy path as a fallback.
15. **Ongoing:** Make the paved road the default for new services; migrate remaining teams; retire the manual runbook once parity is proven.

How I'd measure whether it worked

- **Primary:** DevOps hours/week spent on deployment assistance (target: 6-8 -> under 1). This is the headline metric.
- **DORA signals:** Deployment failure rate and change-failure rate (DORA), MTTR for bad deploys, and deployment frequency/lead time. I built Grafana deploy-status dashboards for exactly this at IDFC, so I'd track these from day one.
- **Adoption:** Percentage of production releases needing manual DevOps intervention, and number of services on the paved road.

Why this one. It attacks recurring DevOps intervention head-on, it compounds (each onboarded team permanently removes toil), and it reuses the exact stack already in place (GitHub Actions + ArgoCD) rather than introducing something new - lowest risk, highest leverage.

Deliverable 4 - Technical Artifact

Below is a trimmed but representative set of artifacts for the Deliverable 3 improvement: (a) a GitHub Actions production-release workflow with the former manual runbook steps encoded as gates, and (b) an Argo Rollouts canary with automated analysis so promotion/rollback is driven by health signals, not a human at a runbook. Illustrative, not production-hardened.

4a. GitHub Actions - gated production release workflow

```
.github/workflows/prod-release.yml
--------------------------------------------------
name: prod-release

on:
  push:
    tags: ["v*"]            # release on semver tag

permissions:
  contents: read
  id-token: write           # OIDC to cloud - no long-lived keys

jobs:
  ci-gates:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Unit + integration tests          # runbook step -> gate
        run: make test

      - name: SCA dependency scan               # runbook step -> gate
        run: trivy fs --exit-code 1 --severity HIGH,CRITICAL .

      - name: Build + scan image
        run: |
          docker build -t "$IMAGE:$GITHUB_REF_NAME" .
          trivy image --exit-code 1 --severity CRITICAL "$IMAGE:$GITHUB_REF_NAME"

      - name: Policy check manifests (OPA)       # runbook step -> gate
        run: conftest test k8s/ --policy policy/

      - name: Validate rendered manifests
        run: helm template ./chart | kubeconform -strict

  promote:
    needs: ci-gates
    runs-on: ubuntu-latest
    environment:
      name: production        # required reviewers = the ONE human approval
    steps:
      - uses: actions/checkout@v4
```

```
      - name: Bump image tag in GitOps repo      # ArgoCD watches this repo
        run: |
          yq -i '.image.tag = strenv(GITHUB_REF_NAME)' gitops/app/values.yaml
          git commit -am "release ${GITHUB_REF_NAME}" && git push
      # ArgoCD + Argo Rollouts take over from here (see 4b)
```

4b. Argo Rollouts - canary with automated analysis

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: web-app
spec:
  replicas: 6
  selector: { matchLabels: { app: web-app } }
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - analysis:                # auto gate - replaces manual smoke test
            templates: [ { templateName: success-rate } ]
        - setWeight: 25
        - pause: { duration: 5m }
        - setWeight: 50
        - pause: { duration: 5m }
        - setWeight: 100
  template:
    spec:
      containers:
        - name: web-app
          image: registry/web-app:TAG
          resources:               # Deliverable 2 fix: measured, not guessed
            requests: { cpu: "250m", memory: "512Mi" }
            limits:   { cpu: "500m", memory: "768Mi" }
          readinessProbe:
            httpGet: { path: /healthz, port: 8080 }
            initialDelaySeconds: 15
            periodSeconds: 10
---
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate
spec:
  metrics:
    - name: error-rate
      interval: 1m
      count: 5
      successCondition: result < 0.02      # < 2% errors or auto-rollback
      failureLimit: 1
      provider:
        prometheus:
          address: http://prometheus.monitoring:9090
```

```
          query: |
            sum(rate(http_requests_total{app="web-app",code=~"5.."}[2m]))
            / sum(rate(http_requests_total{app="web-app"}[2m]))
```

Why this artifact. It shows the mechanism of the fix end-to-end: manual runbook steps become explicit CI gates, the single human approval is an environment protection rule, and production promotion is driven by a Prometheus-backed health check that rolls back automatically. It is intentionally readable, not exhaustive.

Deliverable 5 - Use of AI During This Assessment

The assessment explicitly invites AI use and is interested in how I use it. I treated AI as a fast drafting and review partner, with myself as the engineer accountable for every claim and config that ships.

Where I used AI

- **Structuring & framing:** To structure the response so all six deliverables were answered clearly, and to pressure-test my prioritisation logic (did my ranking rationale have gaps?).
- **Artifact scaffolding:** To produce first-draft YAML/workflow snippets for the GitHub Actions gates and the Argo Rollouts canary, which I then edited for correctness and realism.
- **Hypothesis checklist:** To act as a sounding board on the OOMKill investigation - enumerating hypotheses so I didn't anchor too early on 'it's a leak'.

What I asked it to help with

- "Draft a GitHub Actions workflow that turns manual release-runbook steps into automated gates" and "show an Argo Rollouts canary that auto-rolls-back on Prometheus error rate."
- "List plausible root causes for a gradual 90-minute memory climb ending in OOMKill with no preceding deploy."

One output I accepted / found useful

Accepted: The suggestion to drive canary promotion from an SLO/error-rate analysis step rather than a fixed timer. It sharpened my Deliverable 3 point that automated rollback is only as trustworthy as the detection behind it - which neatly tied Deliverables 2 and 3 together. I kept this.

One output I challenged / corrected

Challenged: An early draft recommended simply raising the memory limit and adding an absolute memory alert as the 'fix' for the incident. I rejected that framing: doubling the limit only delays the same OOM if the cause is a leak, and an absolute threshold is exactly the late-firing alert that already failed us. I corrected it to a leading-indicator alert (memory-as-%-of-limit trend + OOMKill count) plus a real root-cause path (heap dump / working-set analysis). I also corrected an AI-suggested JVM heap setting that ignored container-cgroup awareness - a mistake I've personally debugged on EKS.

How I'd validate AI-generated infra before production

16. **Never trust blindly:** Treat AI output as an untrusted first draft. It never goes straight to prod.
17. **Machine-check it:** Static validation: lint/parse, kubeconform/helm template, terraform validate + plan, and policy-as-code (OPA/Conftest) so generated manifests meet our standards automatically.
18. **Blast-radius control:** Apply in an isolated lower environment first; confirm behaviour, resource usage, and that terraform plan shows exactly the intended diff (no surprise deletes).
19. **Security:** Security review: scan images/dependencies (Trivy), check IAM for least privilege, and confirm no secrets are hard-coded.

20. **Accountable sign-off:** Human review and peer review against our conventions, then promote through the same gated pipeline as any other change.

Principle. AI accelerates the drafting and broadens my hypothesis set; it does not transfer accountability. I verify facts against official docs and validate every config through the same automated gates and review I'd demand of any human-written change.

Deliverable 6 - Slack Update to the VP of Technology

Audience: technical VP who does not need low-level detail. Constraint: max 150 words. Tone: calm, factual, clear on known vs. unknown and on next steps.

Subject: Update on Tuesday's production incident

Hi [VP], a quick update on the recent customer-facing incident.

What happened: application memory grew steadily over ~90 minutes until pods hit their memory limit and were restarted by Kubernetes. During that restart cycle, customers saw elevated errors for roughly 35 minutes. Service is stable now.

What we know: this was a gradual memory exhaustion, not a bad deploy - no release went out beforehand. Increasing the memory limit restored service.

What we don't know yet: the root cause of the growth (likely a memory leak or a workload change). A bigger limit buys time but isn't a fix.

Next: I'm leading a root-cause investigation and adding earlier memory alerts so we detect this before customers do. I'll share findings and a prevention plan by [date].

Word count: ~140 words (excluding subject line).

Appendix - Supporting Artifacts

Additional snippets referenced across the deliverables. Illustrative.

A1. Right-sizing & autoscaling (Deliverable 2 prevention)

VPA in recommendation mode keeps memory/CPU sizing honest; HPA scales on load so spikes become more pods, not fuller memory.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: web-app }
spec:
  scaleTargetRef: { apiVersion: argoproj.io/v1alpha1, kind: Rollout, name: web-app }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 65 } }
---
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata: { name: web-app }
spec:
  targetRef: { apiVersion: argoproj.io/v1alpha1, kind: Rollout, name: web-app }
  updatePolicy: { updateMode: "Off" }   # recommend only; apply via review
```

A2. Leading-indicator memory alert (fixes the late-firing alert)

Fires on memory trending toward the limit and on OOMKills - before customers are impacted - instead of a single late absolute threshold.

```
groups:
  - name: memory-leading-indicators
    rules:
      - alert: PodMemoryNearLimit
        expr: |
          max by (pod) (
            container_memory_working_set_bytes{container!=""}
            / on (pod) kube_pod_container_resource_limits{resource="memory"}
          ) > 0.80
        for: 10m
        labels: { severity: warning }
        annotations:
          summary: "Pod {{ $labels.pod }} > 80% of memory limit for 10m"
      - alert: PodOOMKilled
        expr: increase(kube_pod_container_status_restarts_total[15m]) > 0
             and on (pod) kube_pod_container_status_last_terminated_reason{reason="OOMKilled"} ==
1
        labels: { severity: critical }
        annotations:
          summary: "Pod {{ $labels.pod }} was OOMKilled"
```

A3. Cloud-cost guardrails (Deliverable 1, Rank 2)

- **Tagging as code:** Enforce tagging (team, service, env) via Terraform default_tags + an OPA/SCP policy that rejects untagged resources - so cost is always attributable.
- **Visibility:** AWS Budgets + anomaly detection alerts to Slack; weekly Cost Explorer review by service/tag to find the 25%-vs-8% gap (likely storage growth, data egress, idle/over-provisioned compute, or un-lifecycled S3).
- **Known levers:** Reuse the levers from my ~$30K/month program at IDFC: right-sizing, Spot for fault-tolerant workloads, S3 lifecycle policies, and shutting down idle non-prod.

Closing. My bias throughout is to make the system observable and safe first, remove recurring toil with the stack already in place, and treat cost as a measurable engineering problem - deciding deliberately what to defer rather than trying to fix everything at once.